Amazon M2 Semantic Session-Based Recommender

Large-scale multilingual next-item recommendation

Goal: predict the next product from the current shopping session — without a user profile.

Candidate catalog

1.39M

products

Test sessions

360,625

full-catalog evaluation

Final Recall@20

42.52%

4-layer Semantic SASRec

1. Problem and project scale

What the system predicts, and why the task is difficult

Prediction task

[i1, i2, …, iT] → iT+1

Input: ordered product IDs already seen in the current session.
Output: the next purchased product.

Why it is hard

1.39M candidates

The same intent can appear in six locales and multiple languages. The final target must be retrieved from ~1.4 million products.

Project objective

Full-catalog ranking

Build a reproducible session recommender, compare architectures, and evaluate against the full catalog — not only sampled negatives.

Success criterion

- High Recall@20: the true next product should appear in the recommendation shortlist.
- Good NDCG/MRR: the true product should be ranked as high as possible.
- Fast inference: precompute item vectors and only encode the current session online.

2. Data structure

Amazon M2 contains product metadata and session-level behavioral sequences

PRODUCT TABLE

Product metadata

1,551,057 rows

Key columns:
- id / ASIN
- locale
- title, brand, color, size, model
- material, author, description
- price

The same ASIN can have different localized metadata rows.

SESSION TABLE

Behavioral sessions

3,606,249 sessions

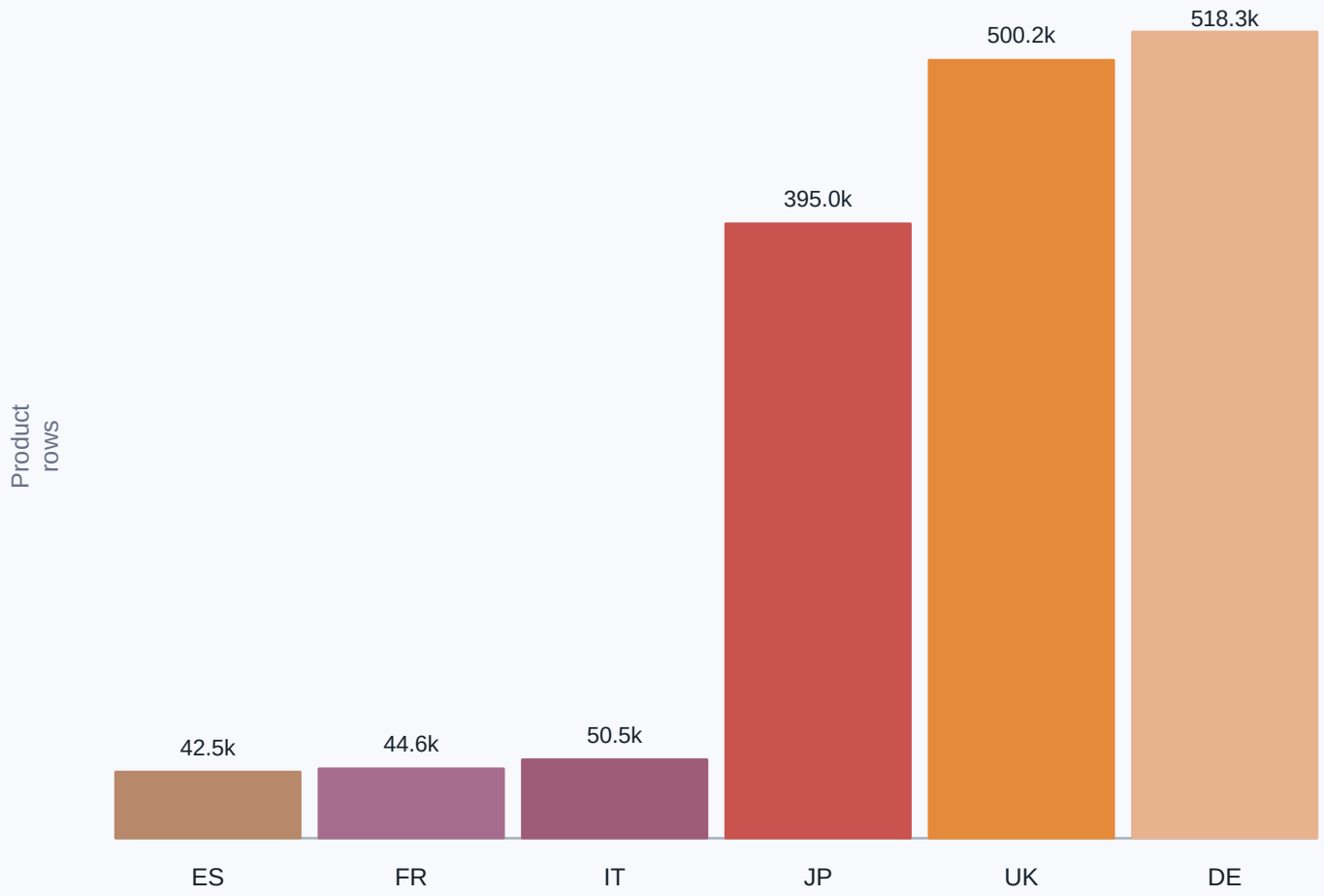
Each row contains:
- prev_items: ordered session history
- next_item: prediction target
- locale

This is a session-based problem: no persistent user profile is required.

Unique ASINs	Behavioral items	Locales	Metadata fields
1,410,675	1,393,956	6	11
		DE, JP, UK, ES, FR, IT	

3. Product EDA: strong locale imbalance

Germany, UK, and Japan dominate the product catalog



Main observation

91.1%

DE + UK + JP account for more than 90% of localized product rows.

Modeling implication

OHE locale

Locale cannot be treated as an ordinal number. It is encoded with one-hot vectors and used as a context signal.

Metadata implication

Multi-representation items

One ASIN may have several localized descriptions. These are embedded separately, then aggregated to one global item vector.

4. Product EDA: price anomaly and multilingual metadata

Cleaning decisions came directly from the data

Abnormal maximum price	Raw median price	Raw mean price	Zero-price products
40,000,000.07	23.28	1,806,107	2,348
repeated across unrelated products	mixed local currencies	dominated by the anomaly	mostly valid digital products

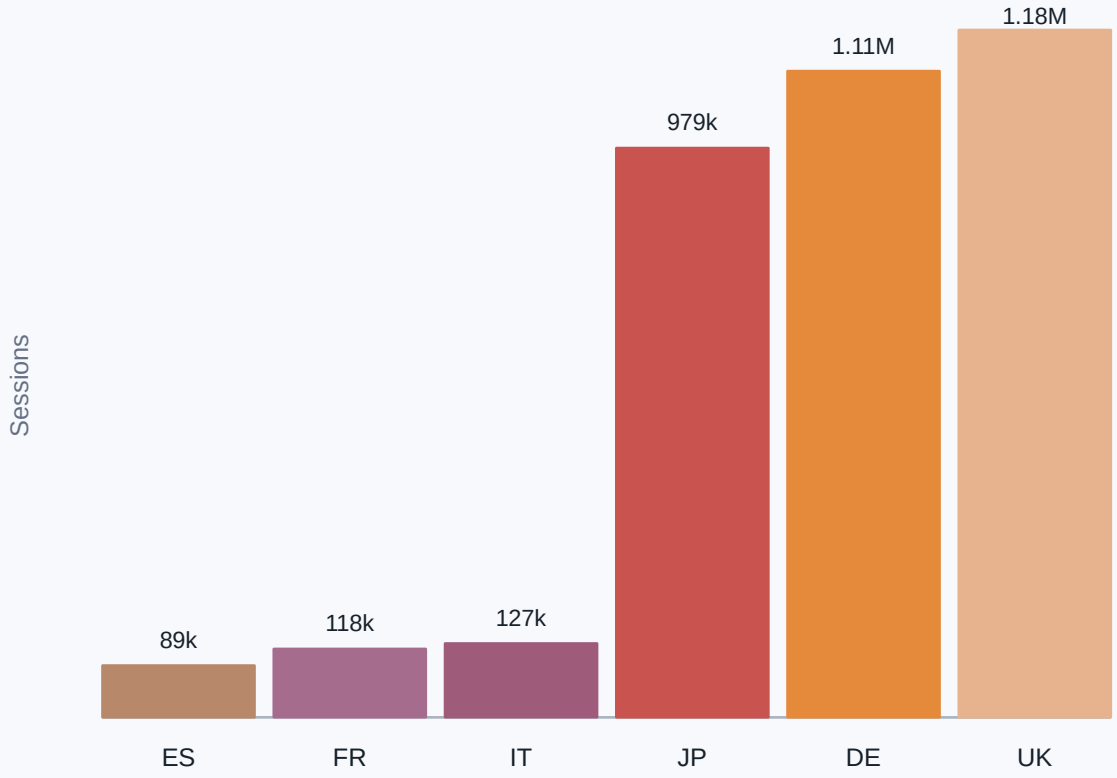
CLEANING DECISIONS

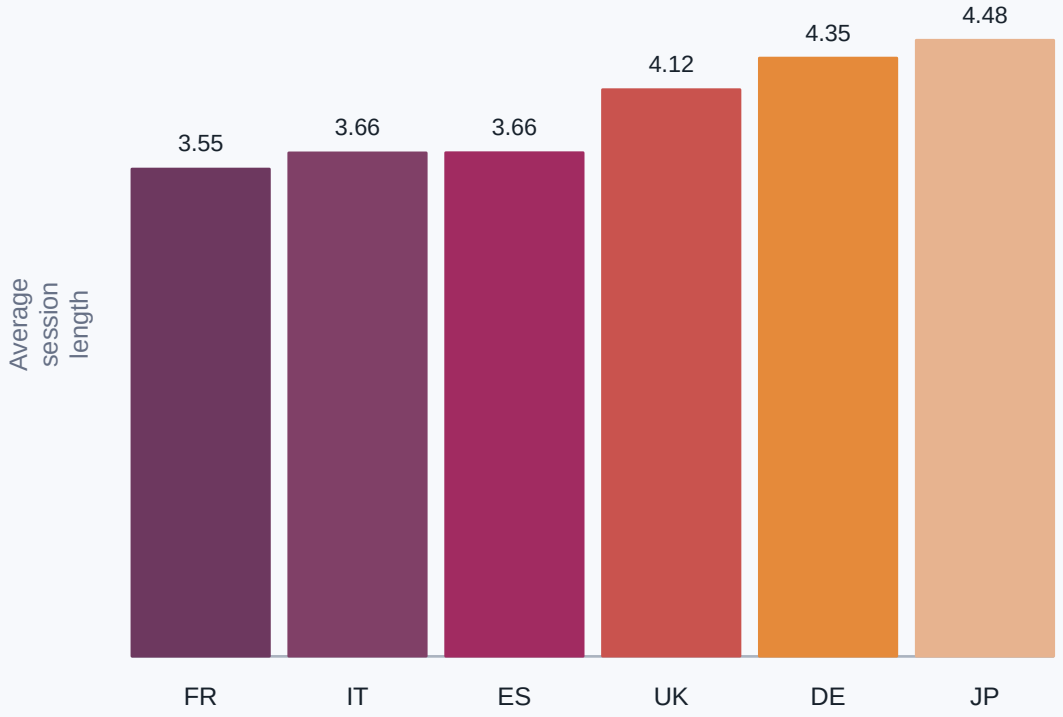
- Replace the 40M sentinel with the median price of the corresponding locale.
- Standardize price separately inside each locale because currencies and scales differ.
- Do not delete zero prices blindly; many are legitimate apps or digital products.
- Keep rich metadata as text because fields such as size, material, and model are multilingual and semantically overloaded.

Key lesson: global statistics were misleading; preprocessing had to be locale-aware.

5. Session EDA: distribution and sequence length

Behavior is also concentrated in UK, DE, and JP





Mean length	Median length	99th percentile	Maximum
4.24	3	18	474

6. Coverage and split strategy

Sanity checks prevented silent evaluation errors

CATALOG COVERAGE



Cold-ish target signal

7.82%

77,883 target items (7.82%) never appeared in session history. Semantic embeddings therefore matter beyond pure behavioral memorization.

FINAL SPLIT

Train	Validation	Test
2,884,999	360,625	360,625
80%	10%	10%

- Shuffle before slicing to avoid a validation set dominated by one locale.
- Keep repeated histories in training; validation and test use unique histories.
- Use the same item-ID vocabulary across train, validation, test, and inference.

7. Preprocessing pipeline

The raw data is converted into model-ready sessions and product representations



Session tensors

Model input

sequence_ids: LongTensor [B, T]
locale_OHE: FloatTensor [B, 6]
target_id: LongTensor [B]

Dynamic padding

Efficient batching

Each batch is padded only to its longest sequence. Padding ID = 0 and is masked in attention.

Frozen product side

Offline embeddings

Text and metadata embeddings are precomputed once and stored on disk. Training does not backpropagate into Voyage.

Result: a lightweight online model, while the expensive multilingual encoder stays offline.

8. Semantic representation of products

Multiple localized descriptions are compressed into one global item vector

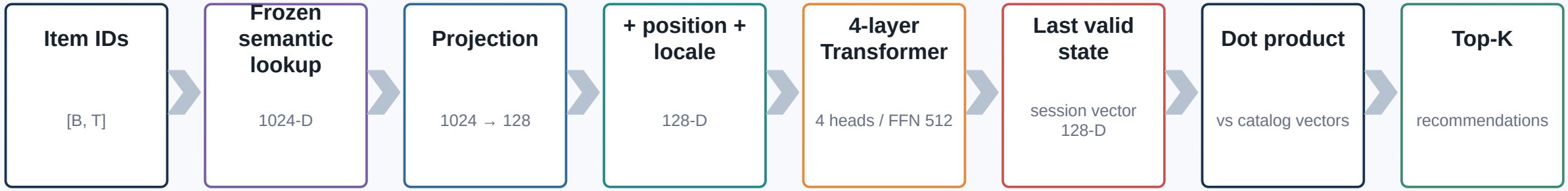


Locale representations	Global item vectors	Embedding dimension	PAD row
1,534,235	1,393,956	1024	0
			global matrix has 1,393,957 rows

Why average first? It avoids repeatedly gathering and combining several locale vectors during every forward pass.

9. Final architecture: 4-layer Semantic SASRec

The Transformer models sequence order; semantic embeddings provide item meaning



Masking

No future leakage

Padding positions are masked. A causal mask prevents a position from seeing future items.

Locale conditioning

Context signal

The 6-dimensional locale OHE is projected to 128 dimensions and added to every item representation.

Retrieval space

Shared 128-D space

Both session and item vectors are normalized. Higher dot product means stronger recommendation relevance.

10. Core mathematics

Only three equations are needed to explain the model

Self-attention

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Each item decides how much information to receive from previous items in the session.

Session-to-item score

$$\text{score}(h,v) = h^T v$$

The last valid Transformer state becomes the session vector h. Candidate item v receives a dot-product score.

Training probability

$$P(i|s) = \exp(\text{score}_i) / \Sigma_j \exp(\text{score}_j)$$

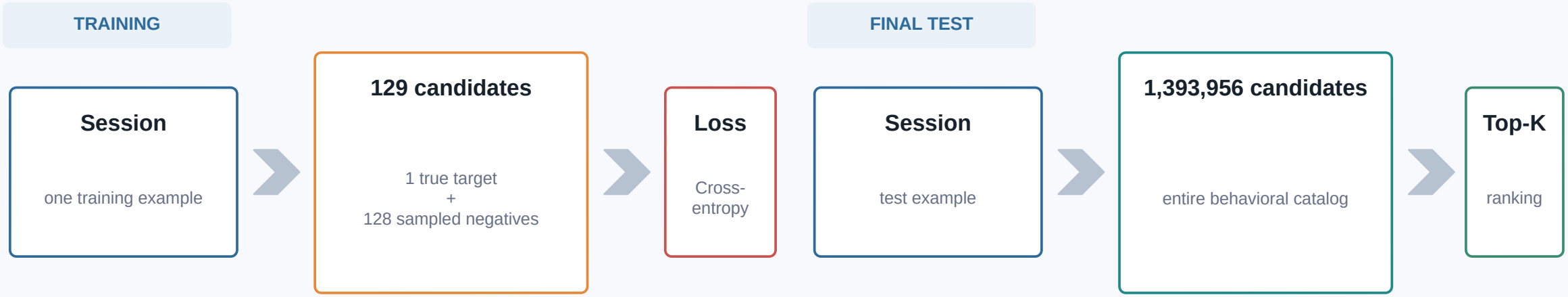
A softmax converts one positive score and sampled negative scores into a classification probability.

Interpretation

- Attention learns which previous products matter for the current intent.
- The projection layer learns which semantic dimensions are useful for recommendation.
- Dot-product retrieval places the session near likely next products in the same learned space.

11. Training: 128 negative samples vs full-catalog test

Training is sampled for speed; final evaluation is exhaustive



Why sample negatives?

16,512 scores / batch at B=128

Scoring all 1.39M items at every optimizer step is unnecessary and extremely expensive. Sampling makes training feasible on one GPU.

Why full-catalog evaluation?

360,625 test sessions

Sampled validation is much easier. The real test asks whether the true product survives competition against the entire catalog.

12. Architecture experiments

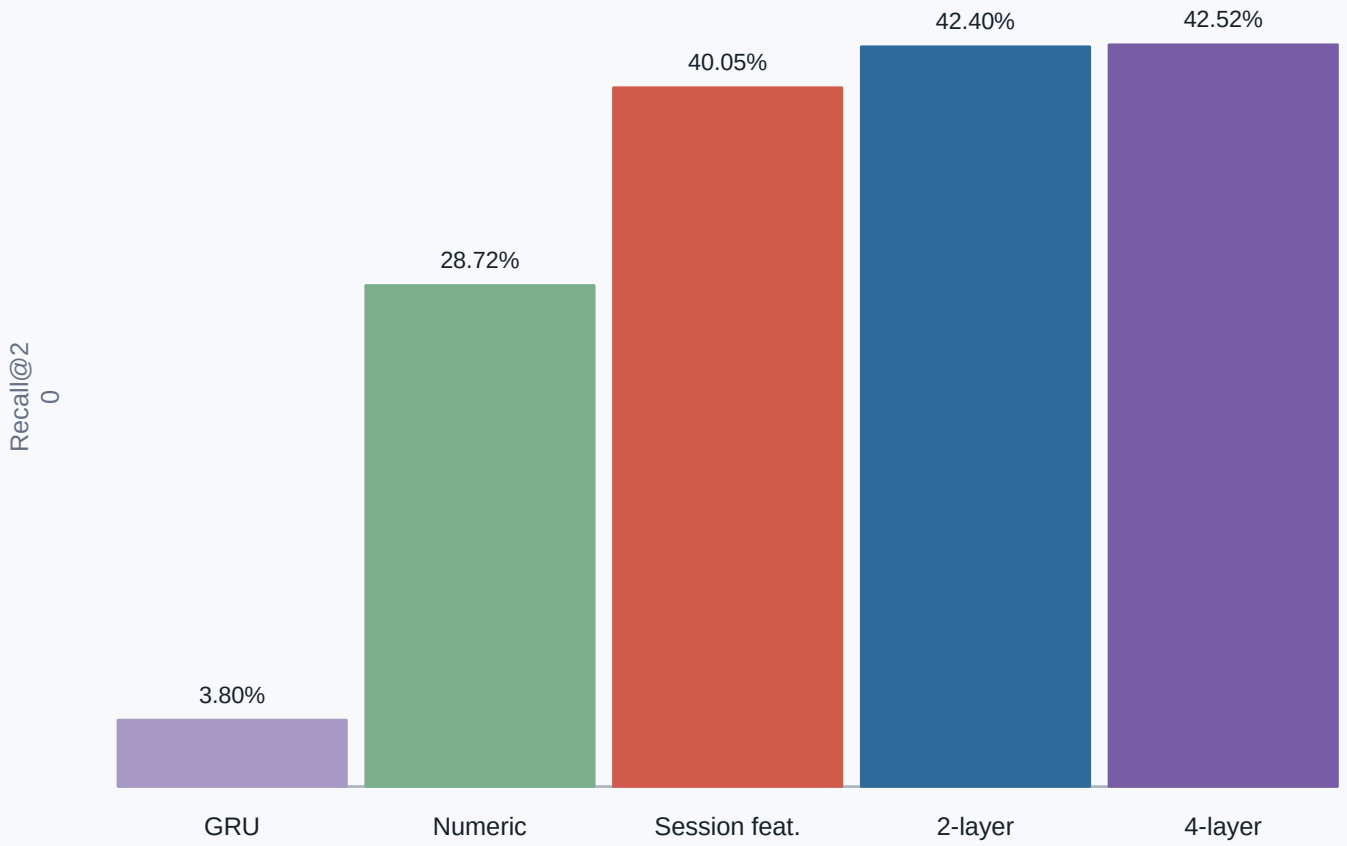
Each experiment changes one modeling assumption

ID	Model	Change	Recall@20	NDCG@20
A	2-layer Semantic SASRec	Semantic projection + 2 Transformer layers	42.40%	0.1932
B	Semantic GRU	Replace Transformer with 2-layer GRU	3.80%	0.0175
C	Semantic + Numeric SASRec	Fuse price + locale product features	28.72%	0.1213
D	Semantic + Session Features	Fuse session length / unique-count statistics	40.05%	0.1821
E	4-layer Semantic SASRec	Increase Transformer depth from 2 to 4	42.52%	0.1932

Main result: sequence architecture matters much more than simply adding more handcrafted features.

13. Full-catalog results

The final 4-layer model achieves the best Recall@20



Recall@1 **2.84%**	Recall@10 **33.52%**
Recall@20 **42.52%**	NDCG@20 **0.1932**
MRR@20 **0.1253**	Catalog **1.39M**

Final evaluation uses every product in the behavioral catalog for every test session.

14. What the ablation study tells us

The simplest explanation is also the strongest one

Transformer vs GRU

42.5% vs 3.8%

GRU collapses on full-catalog retrieval. Self-attention is much better at modeling short, repeated, non-uniform shopping sequences.

Numeric metadata

28.7% Recall@20

Price + locale product features reduce quality. The global item vector already captures strong semantics, while averaging locale-level numeric metadata loses information.

Session statistics

40.1% Recall@20

Length and repetition features help less than expected. The Transformer already observes the sequence structure directly.

Depth: 2 → 4 layers

+0.11 percentage points

The 4-layer model gives the best Recall@20, but the gain is small. This is a diminishing-return result, not a dramatic architectural jump.

Final choice

Best quality / near-tie efficiency

Use 4-layer Semantic SASRec as the final model because it has the highest primary metric. Keep the 2-layer model as the efficiency baseline.

15. Real recommendation examples

The model succeeds when session intent is coherent — and fails when intent is mixed

Case 1 — mixed intent

Failure case

Session contained tablets, cosmetics, Christmas products, a cookbook, and spices.

Model recommendation theme: cooking and spices.

True next item: Barbie Advent Calendar.

Outcome: miss in Top-10.

Interpretation: the model followed the recent local intent and ignored an older competing interest.

Case 2 — cable labeling

Rank 1

Session contained a DYMO label printer and cable-label products.

True next item: cable labeling pack.

Top recommendation: the exact target.

Outcome: Rank 1.

Interpretation: semantic similarity and sequential intent agree strongly.

Case 3 — Jurassic World

Rank 4

Session contained several editions of Jurassic World.

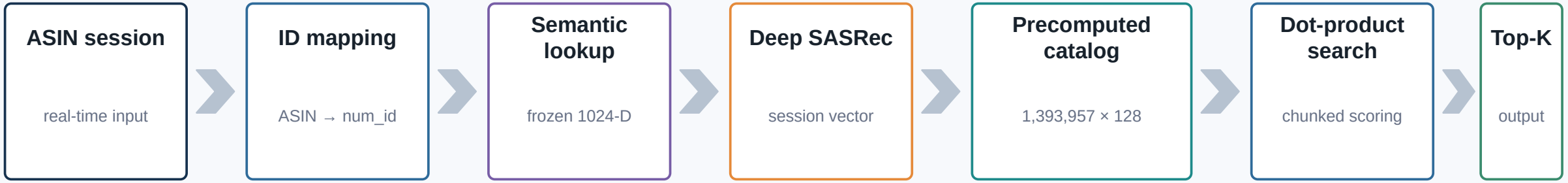
The recommendation list was dominated by closely related movie editions.

True target appeared at Rank 4.

Interpretation: the semantic embedding space forms a meaningful product neighborhood.

16. Inference and production path

Most expensive work is moved offline



NEXT PRODUCTION IMPROVEMENTS

- Replace brute-force dot-product search with FAISS / ANN candidate retrieval.
- Apply locale-availability filtering before final ranking.
- Mine hard negatives instead of purely random negatives.
- Add a second-stage reranker for the top 100–500 candidates.

17. Conclusion

Three points to remember

Final model

4-layer SASRec

semantic + sequential

Full-catalog Recall@20

42.52%

1.39M candidates

Test set

360,625

sessions

1. Data quality mattered

Locale imbalance, duplicate histories, abnormal prices, multilingual metadata, and variable sequence length all changed the modeling decisions.

2. Semantics + sequence won

Frozen multilingual item embeddings provide meaning; SASRec learns how that meaning evolves across the current session.

3. Full-catalog evaluation mattered

Sampled metrics looked much easier. The final result is credible because the model was tested against the entire 1.39M-item catalog.

Final takeaway: a lightweight Transformer can convert a multilingual shopping session into a useful retrieval vector at million-item scale.